

ANCHOR: Asset-Informed Geometric Skills for Language-Conditioned Manipulation

Anonymous Authors

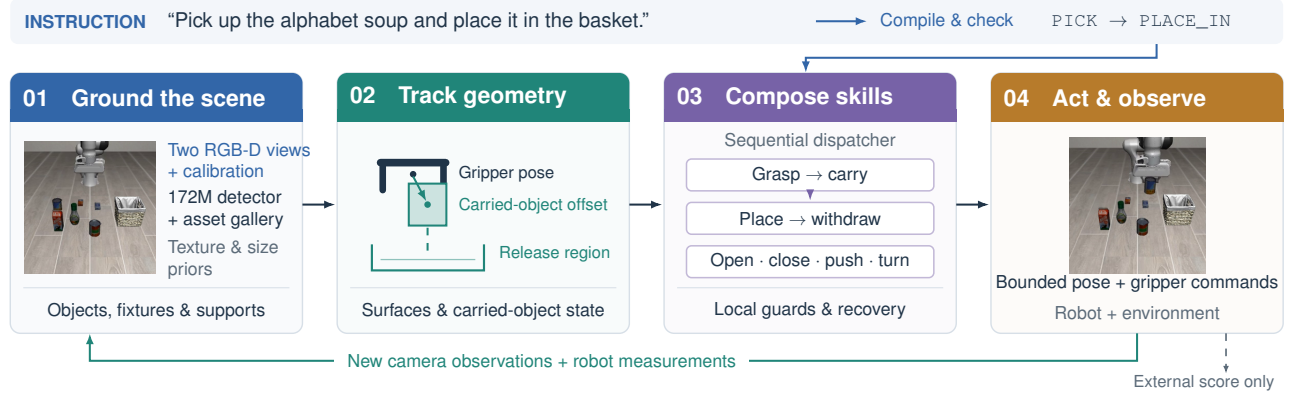


Fig. 1. From language to physical interaction with ANCHOR. Supported instructions specify a skill sequence; calibrated RGB-D and asset priors provide the geometry used to execute it. The controller carries object and fixture state across stages and advances using local observation checks. The frozen 172M-parameter detector is the only neural model. Insets show an actual Object 00 rollout; the geometry illustration is schematic. External benchmark success is recorded separately from policy execution.

Abstract—Multi-stage manipulation depends on the geometry left by each interaction: a grasp determines the carried-object pose, release changes contact, and opening a fixture changes the next approach. We present ANCHOR, an asset-informed manipulation system that makes these dependencies explicit in geometric skills. A deterministic compiler maps supported instructions to skill sequences, grounded by a frozen 172M-parameter detector, two calibrated RGB-D views, and public asset priors. The controller maintains carried-object offsets, fixture geometry, and local transition checks, without training an action policy on robot demonstrations. A frozen controller achieves 360/400 (90.0%) external ever-success across 40 LIBERO tasks. Targeted skill revisions improve microwave execution from 0/10 to 8/10 and drawer execution from 3/10 to 9/10, with unchanged neural weights. Combining the 20 retest episodes with 380 retained episodes yields 374/400 (93.5%) across 3 controller versions; this is not a full-suite evaluation of the latest controller. Separate controller and evaluator outcomes expose 44 disagreements. The study characterizes a compact perception model coupled with inspectable geometric control under known-asset and calibrated-sensing assumptions.

I. INTRODUCTION

Consider the instruction to place a mug inside a microwave and close it. Identifying the mug and appliance determines what to manipulate, but leaves several physical decisions unresolved. The mug must enter the cavity in its grasped configuration, separate from the fingers on release, and remain inside as the hand withdraws and engages the door. An error at any transition can invalidate the next skill even when the object labels and high-level sequence are correct. This dependence on the state left by previous interactions is a central challenge in executing multi-stage language instructions.

Existing approaches connect language to action through several interfaces. Vision-language-action (VLA) models learn action prediction from robot demonstrations [1]–[3]. Program-based systems expose perception and control calls [4], while geometric approaches express desired behavior through spatial value maps or keypoint constraints [5], [6]. These approaches motivate a practical design question for environments with known assets: how much manipulation capability can a compact perception model support when geometric skills explicitly represent the state needed by subsequent interactions?

We present ANCHOR (Fig. 1), a language-conditioned tabletop manipulation system built around this design. A deterministic compiler converts supported instructions into ordered skills. A frozen 172M-parameter Grounding DINO detector, together with an asset gallery and two calibrated RGB-D views, grounds the referenced objects and fixtures. Geometric controllers then generate bounded Cartesian increments and gripper commands. The system uses no robot demonstrations to train an action policy; its manipulation knowledge is supplied through public asset geometry and manually designed skills.

The design connects perception to execution through three kinds of state. Observed surfaces and asset dimensions determine contact and placement regions. An estimate of the object-to-gripper displacement translates an object goal into an end-effector target. Local checks on pose, gripper width, visual geometry, and contact evidence determine when execution can advance. For the microwave example, these quantities distinguish reaching a placement pose from releasing the mug and clearing the opening. The same explicit

targets and checks are retained in execution traces, so a failed stage can be examined in terms of the geometry and measurements that drove it.

We characterize this system on the Spatial, Object, Goal, and Long suites of LIBERO [7], using ten official initial states for each of 40 tasks. The evidence consists of one complete frozen-controller evaluation and two subsequent task-specific revisions. The latter address microwave release and closure, and the transition from drawer opening to bowl manipulation. They illustrate changes to geometric execution with unchanged detector weights. We make three contributions:

- 1) An asset-informed manipulation system that combines one frozen 172M-parameter detector with geometric action generation, with explicit sensing, prior-knowledge, and language requirements.
- 2) A documented integration of carried-object geometry, fixture estimates, and local transition checks across grasp-place and contact skills, with two complete task retests illustrating targeted revision.
- 3) A record-based evaluation of task outcomes, neural model requirements, and failure states that distinguishes controller completion from external benchmark success and preserves controller-version provenance.

The contribution is an integrated system study using established geometric and state-machine techniques. Its scope is manipulation with known assets, implemented instruction forms, and calibrated sensing. The task-specific revisions are development results, and their combined statistics are reported separately from the original full evaluation.

II. RELATED WORK

A. Geometric Representations for Manipulation

Task and motion planning couples discrete action choices with continuous geometric feasibility [8]. The representation used at this interface determines which physical relations the controller can express. kPAM uses semantic 3-D keypoints to specify manipulation goals as geometric costs and constraints [9]. VoxPoser grounds language in spatial value maps for motion planning [5], and ReKep optimizes sequences of relational keypoint constraints [6]. ANCHOR shares the use of explicit geometry between perception and action. Its representation consists of object bounds, support and fixture geometry, and carried-object offsets, consumed by specialized finite-state skills. This fixed skill structure defines its coverage; it does not optimize a general task-and-motion problem.

B. Language and Executable Skills

SayCan combines language-model proposals with learned skill affordances to select executable actions [10]. Code as Policies generates programs that process perceptual outputs, express control logic, and call robot interfaces [4]. These systems use language models to construct behavior. ANCHOR uses a deterministic grammar and a checked skill vocabulary, binding spatial references to observed entities. This choice provides a fixed program whose geometric

targets and transitions can be inspected, while restricting instructions to the supported constructions. Our focus is the physical execution of these programs and the state passed between their skills.

C. Pretrained Perception and Learned Action Policies

Grounding DINO provides text-conditioned object proposals [11]. ANCHOR combines its frozen detector with calibrated depth and asset-derived texture and size priors. OpenVLA [1] and SmolVLA [2] learn robot action generation from demonstrations, with SmolVLA emphasizing a compact architecture. OpenVLA-OFT studies design choices for faster and more successful VLA execution [3]. These methods provide context for the model requirements and reported task outcomes of our system. Section V-B distinguishes their sensing, training, and evaluation conditions. In particular, ANCHOR exchanges action-policy training for asset knowledge and manual skill development; pretrained perception remains part of the system.

III. METHOD

ANCHOR executes a supported instruction by grounding its entities, constructing geometric targets, and checking the physical transitions between skills. Figure 1 summarizes this data flow, and Table I lists the information available to the system.

A. Task and Information Interface

Given an instruction ℓ , the controller produces actions $a_t = \pi(o_{\leq t}, \ell; \mathcal{P})$, where $\mathcal{P}$ contains fixed prior information and the observation at step t is

$$o_t = (\{I_t^c, D_t^c, K^c, T_{WC,t}^c\}_{c \in \{a,w\}}, r_t). \quad (1)$$

Here a and w denote the fixed and wrist cameras, I is RGB, D is metric depth, K is calibration, and T_{WC} maps camera coordinates into the world frame. Robot measurements r_t include end-effector pose, joint state, gripper state, and available force/torque measurements. The grasp-place adapter uses pose, joint positions, and gripper width; contact execution can access the broader robot measurement interface.

Depth, camera calibration, and robot measurements are supplied by the simulator adapter. The policy observation types exclude object pose labels, segmentation IDs, BDDL predicates, reward, and task-success flags. Task identifiers and initial-state indices remain with the evaluator; reset passes the instruction and a nonsemantic episode token.

The prior collection $\mathcal{P}$ contains object texture histograms, dimensions derived from public collision geometry, robot geometry, class-specific handling rules, and motion thresholds. These priors specify how to interpret observations and parameterize skills. The neural detector is pretrained; the action controller is designed manually. No action-policy fitting or robot-demonstration loading occurs in the evaluation.

B. Constrained Language Compilation

The compiler maps supported language to

$$\mathcal{G}(\ell) = (s_1, \dots, s_M), \quad s_j = (k_j, e_j^{\text{src}}, e_j^{\text{dst}}, \rho_j). \quad (2)$$

Here k_j is a skill, the entity bindings specify source and destination, and ρ_j represents a placement relation or instance selector. The vocabulary contains pick, place-on, place-in, relative placement, stack, open, close, push, turn-on, and turn-off. For example, opening a drawer and placing a bowl inside produces an articulation segment followed by pick and place-in steps. A two-object basket instruction produces two successive pick/place pairs.

Bindings retain relations such as *next to*, *between*, or an ordering among visible instances. They do not encode simulator object identity. Before motion, the compiler checks skill arguments and held-object consistency, including whether each place step refers to the object acquired by a preceding pick. Physical executability is assessed during skill execution. The grammar covers known LIBERO phrases and an allow-listed entity vocabulary; no language-model resolver is configured.

C. Asset-Informed Visual Grounding

The deployed Grounding DINO Tiny checkpoint contains 172,249,090 unique parameters, including its visual and text encoders. All weights are frozen, and one detector instance serves both camera views in each evaluation process. The asset gallery, language compiler, and geometric controllers add no neural model parameters. The pinned checkpoint and parameter audit are included in the manuscript artifacts.

Each calibrated pixel $\tilde{u} = (u, v, 1)^\top$ with valid depth contributes

$$x_W = R_{WC}(D(u, v)K^{-1}\tilde{u}) + p_{WC}. \quad (3)$$

Here R_{WC} and p_{WC} are the rotation and translation of T_{WC} . Workspace filtering, support estimation, and connected components produce candidate point sets. Grounding DINO supplies additional language-conditioned boxes, which are converted to 3-D hypotheses using depth. These proposals are combined with observed bounds and asset-derived texture and size evidence. The gallery compares normalized HSV histograms using

$$b(h, h_k) = \sum_i \sqrt{h_i h_{k,i}}. \quad (4)$$

Its classification score combines color and shape terms, with configured weights 0.4 and 0.6, a softmax temperature of 0.13, and additional class-dependent footprint gates. This combination associates visual proposals with known asset classes without an oracle object mask.

Cross-camera hypotheses with matching labels are clustered in 3-D. For a cluster, the implemented fusion weights are proportional to $\max(q_i, 0.05)\sqrt{n_i}$, where q_i is confidence and n_i is point count. The default center-distance threshold is 4.5 cm. The resulting state distinguishes observed bounds and supports from geometry completed using size priors. A prior can stabilize the center of a partly visible object without establishing that an unobserved region is free.

Geometric selectors bind instruction references to candidate instances. At selected phase transitions, fresh views are checked against previous estimates to reject abrupt changes caused by hand occlusion. Nearby repeated objects can still produce ambiguous associations. Fixture routines further refine the selected geometry into support surfaces and opening regions.

D. Carried-Object Geometry and Placement

Consecutive manipulation skills share a grasp-place controller. Its phases include acquisition, pregrasp motion, approach, finger closure, retention checking, lift, transport, descent, release, and retreat. Transitions combine pose tolerances, measured gripper width, visual evidence, and bounded retry logic. Different grasp modes account for exterior pinch, rim pinch, and interior expansion; mode choice depends on the object and observed accessibility.

The chosen grasp also determines the geometry used during placement. In particular, the gripper and carried object generally have different centers. Let $\hat{d}_t$ be the estimated world-frame vector from the end effector to the carried-object reference point. For desired object location p_o^* , placement uses

$$p_{ee}^* = p_o^* - \hat{d}^*, \quad \hat{d}^* = \Delta R \hat{d}_t \quad (5)$$

when the planned wrist motion rotates the held geometry by ΔR . Centering the gripper at p_o^* would therefore displace the object from its intended location. The implementation supplements Eq. (5) with support, release-height, aperture, and fixture-specific corrections. The offset is estimated from precontact geometry or refreshed RGB-D evidence; its origin is retained in the execution trace.

Placement must also leave room for the hand to withdraw. Microwave entry aligns the carried object with the cavity width and uses a lateral approach. Repeated basket placements use separate diagonal landing centers. Before basket withdrawal, the controller checks measured jaw clearance and can apply a bounded wrist tilt to clear a tall object near the palm. Release is followed by a geometric observation check before the program advances.

E. Fixture Geometry and Contact Skills

The dispatcher separates consecutive grasp-place steps from atomic contact skills. Drawer and door interactions use observed fixture geometry to construct approach, engagement, motion, release, and retreat stages. Knob and push skills use their own target detectors and motion templates. Shared episode state includes an initial robot-pose reference and, where applicable, an observed drawer anchor for subsequent placement.

Grounded object boxes are refined into surfaces needed for physical interaction. A complete visible burner disk anchors the stove support center; public plate thickness supplies the offset to the load-bearing surface. That unoccluded estimate is retained across sequential placements. For an open microwave, visible vertical control-panel and side-wall surfaces, together with the roof height, constrain the cavity and hinge frame through public physical dimensions. A flat

tabletop crop or a roof without supporting vertical structure is insufficient for this fit.

For microwave closure, the revised skill estimates a vertical door panel within the observed hinge sweep. Initial fitting requires panel evidence outside the appliance body to exclude the fixed front frame. The gripper approaches the exterior panel face and follows bounded hinge-frame waypoints. A fresh panel-angle estimate after a motion plateau can initiate withdrawal; a second view after retreat checks closure. Section V-C describes this revision together with the preceding mug-release check.

Contact guards combine position error, measured progress, and changes in end-effector force. One guard requires a minimum contact duration and bounded position error, plus either stalled error reduction or sufficient force change. This provides evidence of contact; a subsequent visual check assesses the resulting fixture configuration. The separation allows the controller to distinguish a mechanical stop from its estimate of task completion.

F. Action Generation and Transition Checks

For a geometric target pose (p^*, R^*) and gripper command g_t , the basic Cartesian servo emits

$$a_t = \text{clip}_{[-1,1]} \begin{bmatrix} (p^* - p_t)/s_p \\ \text{Log}(R^* R_t^\top)^\vee / s_R \\ g_t \end{bmatrix}. \quad (6)$$

The rotation term is the axis-angle error between the current and target orientations. The general grasp-place scales are $s_p = 0.05$ m and $s_R = 0.25$ rad; cavity handling can select another rotation scale. Nominal pose tolerances of 8 mm and 0.12 rad gate transitions between motion phases. Specialized branches additionally bound translation or rotation during constrained motions.

Phase counters and geometric checks bound local recovery. The controller may release a rejected grasp, reacquire a visible handle, or execute a short clearance motion after stalled drawer interaction. Each stage supplies explicit targets, observations, and stopping reasons for inspection. These checks govern execution using the policy’s own measurements. The benchmark predicate is unavailable to both action selection and stopping, so local completion remains a separate outcome, evaluated in Section V-D.

IV. EXPERIMENTAL PROTOCOL

A. Tasks, Initial States, and Runtime

The evaluation contains $4 \times 10 \times 10 = 400$ episodes: Spatial, Object, Goal, and Long, ten tasks each, and official initial-state indices 0–9. Long is named `libero_10` in the implementation. Global random generators and the environment configuration use seed 7. The environment adapter preserves or replays the task reset stream when starting at a later initial-state index, because resetting can affect fixture configuration beyond the saved joint state.

We use two 256×256 RGB-D cameras and a Panda with operational-space pose control. A ten-step settling period

TABLE I
DECLARED CONDITIONS FOR THE EVALUATED SYSTEM.

Property	Setting
Policy observations	Fixed and wrist RGB-D, camera calibration, robot measurements
Offline priors	Public asset textures/dimensions, robot geometry, hand-written skill rules
Action generation	Geometric state machines and clipped Cartesian increments
Neural model	One frozen Grounding DINO Tiny (172M parameters)
Action-policy training	None; skills and thresholds are designed manually
Scene truth in policy	No object pose labels, segmentation IDs, or BDDL predicates in observation types
Success feedback	External latch; no score-driven policy stopping
Coverage	Known benchmark assets and implemented instruction grammar

precedes scored actions. The Spatial/Object/Goal/Long action budgets are 220/280/300/520, matching the numerical budgets in the OpenVLA evaluation script [12]. Our environment seed and success-handling logic differ from that script; its environment utility uses seed 0. The original full run uses Ubuntu 22.04, Python 3.12, PyTorch 2.11.0 with CUDA 12.8, MuJoCo 3.3.2 [13], robosuite 1.4.0 [14], and hf-libero 0.1.4 on one RTX 5090. The targeted retests use the same software environment on an RTX 4090.

The original full campaign records 360/400 successes with one frozen controller. We subsequently revise Long 09 microwave execution, then Goal 03 drawer execution. Each task is retested on all ten official initializations, in index order, with one frozen source snapshot. The retests record 8/10 and 9/10 successes, respectively. The combined statistics retain 380 original episodes for the other 38 tasks and replace both complete task records, including failures. They contain 3 controller versions and do not establish a full-suite result for the latest controller. The original campaign, all source manifests, and each record’s source version are preserved. These initializations informed development; the retests are not held out.

B. External Scoring and Internal Completion

Let q_t be the official success predicate observed by the evaluator. With policy-controlled stopping time τ capped at budget H , the recorded outcome is

$$Y = \bigvee_{t=1}^{\tau} q_t \vee q_{\text{final}}, \quad \tau \leq H. \quad (7)$$

The final query also handles a zero-action stop. Reward and success do not enter the policy observation. The evaluator continues after a success signal until the controller requests stopping or the action budget expires. The score therefore represents *ever reaching* the predicate during this rollout, not necessarily satisfying it at the final frame.

Each record additionally stores terminal controller state, step count, elapsed time, failure message, and geometric traces. The reported success rate is $\hat{p} = N^{-1} \sum_i Y_i$. Equal task sample counts make the overall episode average equal to

TABLE II

OBSERVED SUCCESS AND EXECUTION COST. SR USES EXTERNAL EVER-SUCCESS. THE COMBINED RECORDS RETAIN 380 HISTORICAL EPISODES AND USE 20 TARGETED RETEST EPISODES. [†]MEDIANS POOL SOURCE-RUN ELAPSED TIMES; HARDWARE AND CONCURRENCY DIFFER.

Suite	Success	SR (%)	Mean steps	Median s [†]
Spatial	93/100	93.0	147.1	68.1
Object	98/100	98.0	195.3	59.5
Goal	97/100	97.0	179.9	66.2
Long	86/100	86.0	324.5	93.4
Overall	374/400	93.5	211.7	68.1

the macro-average over tasks in the completed run. Each reported batch retains all planned failures; candidate outcomes are not mixed within a task. Controller-internal retries remain within one episode.

C. Scope of the Evidence

All three source runs pass their respective coverage checks. The combined record set contains 400 unique task/initial-state pairs with the declared configuration; original record hashes identify the retained and replaced episodes independently. Tables are generated from hashed original records. Published values are stored separately with their cited sources. Per-task counts expose the sample size behind each rate.

V. RESULTS AND ANALYSIS

A. Task Outcomes across the Four Suites

The original full evaluation records 360/400 successes (90.0%) with one frozen controller. Replacing the complete Long 09 and Goal 03 task records with their respective retests yields 374 successes in 400 records (93.5%; Table II). This combined summary contains 380 retained episodes and 20 retest episodes from 3 controller versions. It describes the available task records, not a full-suite measurement of the latest controller. The suite rates are 93.0% for Spatial, 98.0% for Object, 97.0% for Goal, and 86.0% for Long.

The task distribution reveals variation hidden by the overall rate (Fig. 2). 24 tasks have ten successful recorded trials. Long 09, placing a mug inside a microwave and closing it, records 8/10; Long 08, placing both moka pots on the stove, records 7/10. These tasks require confined access, sequential placement, or fixture interaction. Drawer-related tasks also differ: Goal 03, opening the top drawer and placing a bowl inside, records 9/10; Spatial 04, retrieving a bowl from a top drawer, records 7/10; and Long 03, placing a bowl in a bottom drawer and closing it, records 7/10. The variation motivates examining individual execution stages rather than attributing all failures to a task category.

B. Model Requirements and Published Results

Table III separates neural model size from the use of robot demonstrations for action generation. ANCHOR has 0.172B parameters in its frozen detector and no learned action network. The FP32 weight file occupies 689 MB (decimal).

TABLE III

MODEL SIZE AND ROBOT-DEMONSTRATION USE FOR ACTION-POLICY TRAINING. VLA SIZES ARE ROUNDED VALUES FROM THE CITED PAPERS; ANCHOR COUNTS ALL UNIQUE PARAMETERS IN ITS DEPLOYED DETECTOR. “NONE” REFERS TO ACTION-POLICY TRAINING, NOT TO PERCEPTION PRETRAINING OR MANUAL SKILL DEVELOPMENT.

Method	Parameters	Policy training
SmolVLA [2]	0.45B	Robot demos
OpenVLA [1]	7B	Robot demos
OpenVLA-OFT [3]	7B	Robot demos
ANCHOR	0.172B	None

The reported SmolVLA model is approximately 2.6 times larger, and the 7B OpenVLA model class is approximately 40.6 times larger. Our count includes both visual and text encoders; the VLA sizes are rounded descriptions from the source papers.

The training distinction concerns robot actions. ANCHOR uses pretrained perception and manually developed skills. The selected SmolVLA simulation model starts from a VLM without robotics pretraining but trains its action policy on LIBERO demonstrations [2]. Parameter counts therefore describe the neural component of each system; the asset priors and engineering effort in ANCHOR remain separate requirements.

Table IV places these requirements alongside published LIBERO success rates. External rows retain the source papers’ reported means and averages; ANCHOR is the only system evaluated in this study. The OpenVLA study uses filtered demonstrations [1]. SmolVLA trains across all 40 tasks and evaluates ten trials per task [2]; OpenVLA-OFT uses filtered demonstrations and evaluates 500 trials per suite in the selected setting [3]. Our system uses calibrated RGB-D, public asset priors, ten official initializations per task, and the ever-success rule in Eq. (7).

These sensing, training, and protocol differences preclude a controlled ranking. OpenVLA-OFT reports a higher average than our combined record set. The comparison locates ANCHOR as a system with a smaller neural component and geometric action generation under stronger explicit priors. It does not measure relative latency or memory consumption. Long is the lowest-scoring suite for every listed method, including ANCHOR; the following cases examine how individual stages affect two multi-stage tasks in our implementation.

C. Tracing Failures to Skill Revisions

The two targeted revisions expose the connection between stored geometric state and controller behavior. Each retest uses all ten original task initializations under one frozen source snapshot, with unchanged action budgets, scoring, and detector weights.

a) *Microwave placement and closure (Long 09)*: The original controller records 0/10. Inspection identified a mug rim remaining hooked on a finger during withdrawal, despite the release command. The revised placement stage waits for

Spatial	10/10	8/10	10/10	9/10	7/10	9/10	10/10	10/10	10/10	10/10
Object	10/10	9/10	10/10	10/10	9/10	10/10	10/10	10/10	10/10	10/10
Goal	10/10	10/10	10/10	9/10	10/10	10/10	8/10	10/10	10/10	10/10
Long	10/10	9/10	9/10	7/10	8/10	9/10	9/10	10/10	7/10	8/10
	00	01	02	03	04	05	06	07	08	09
Official task index (within each suite)										

Fig. 2. External successes per ten trials. Columns identify different tasks in each suite; darker cells indicate higher success. Goal 03 and Long 09 use complete targeted retests; other cells retain the original full-campaign records.

TABLE IV

LIBERO SUCCESS RATES (%). PUBLISHED ROWS CONTAIN THE REPORTED MEANS FROM OPENVLA TABLE 12 [1], SMOLVLA TABLE 2 [2], AND OPENVLA-OFT TABLE I [3]; THEY ARE NOT RERUNS BY US. OPENVLA-OFT USES ITS FULL SETTING. THE FINAL ROW IS MEASURED IN THIS WORK. IT COMBINES 380 HISTORICAL EPISODES AND 20 TARGETED RETEST EPISODES FROM 3 CONTROLLER VERSIONS; IT IS NOT A FULL RERUN OF THE LATEST CONTROLLER. ALL METHODS RECEIVE LANGUAGE; THEIR SENSING, TRAINING, AND PROTOCOLS DIFFER.

Method	Sensing and action-policy setup	Spatial	Object	Goal	Long	Average
<i>Published results (reported means)</i>						
Diffusion Policy [1]	Single RGB; policy trained from scratch per suite	78.3	92.5	68.3	50.5	72.4
Octo [1]	Single RGB; pretrained policy fine-tuned per suite	78.9	85.7	84.6	51.1	75.1
OpenVLA [1]	Single RGB; pretrained policy fine-tuned per suite	84.7	88.4	79.2	53.7	76.5
SmolVLA (0.45B) [2]	RGB/state; VLM-initialized policy trained across suites	90.0	96.0	92.0	71.0	87.3
OpenVLA-OFT [3]	Two RGB views + state; policy fine-tuned per suite	97.6	98.4	97.9	94.5	97.1
<i>Measured in this work</i>						
ANCHOR	Two RGB-D views + state; asset priors, geometric skills	93.0	98.0	97.0	86.0	93.5

a measured 78-mm jaw opening and a settling dwell before withdrawing. The subsequent close skill uses the observed door panel and bounded hinge-frame waypoints described in Section III. Its motion and closure checks refer to the exterior panel face. The complete retest records 8/10. The two remaining failures occur during grasp approach and cavity-entry staging, before completion of the placement-and-closure sequence.

b) From drawer opening to bowl placement (Goal 03): The original controller records 3/10. Execution traces show prolonged motion after handle release, including wrist and elbow joint-limit encounters on the approach to the bowl. The revision compares equivalent jaw frames along a continuous inverse-kinematics path using the public robot model, and bounds simultaneous translation and rotation. During transport, it also moves the held bowl beyond the observed drawer front before lifting. The complete retest records 9/10, with seven previous failures becoming successes and one previous success regressing.

Both cases modify the physical transition between otherwise supported skills. Their traces expose the target, the measured state, and the local check at the point of failure, making the revision concrete without changing the perception model. Each result evaluates a bundle of changes on initializations used during development; the retests do not

isolate the effect of each change or measure performance on held-out tasks. Figure 3 supplies historical visual context: a successful Object 00 rollout and an unsuccessful Goal 03 rollout from the original campaign, before these revisions.

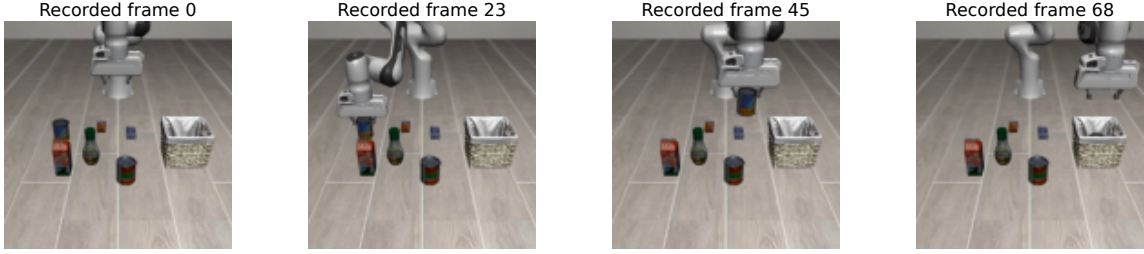
D. Local Completion and External Success

Controller completion and benchmark success measure different events. Table V crosses the terminal controller state with the external ever-success label. In 7 episodes the controller reports completion but the evaluator reports failure. Conversely, 37 episodes reach external success without controller completion. The total disagreement is 44/400 (11.0%).

The latter group includes a predicate becoming true before a later local failure or budget exhaustion. A local check may continue to demand a geometric relation after the benchmark has already registered success. Thus the recorded score answers whether the predicate was ever reached during the rollout; it does not establish that the controller recognized success or preserved it until stopping. Substituting local completion for the benchmark label would change the measured outcome. Feeding benchmark success back to the policy would change the information available for stopping.

The records retain neither first-success time nor a separate final-state label. They therefore cannot determine how continued execution changes final-state success or how much runtime an early-stop evaluator would save.

Object 00 / init 00: external success



Goal 03 / init 02: action-budget exhaustion

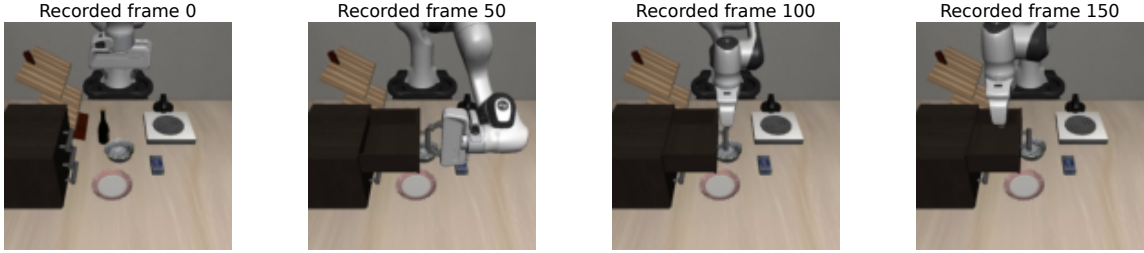


Fig. 3. Actual fixed-camera frames. Top: *pick up the alphabet soup and place it in the basket* (Object 00, initial state 00). Bottom: *open the top drawer and put the bowl inside* (Goal 03, initial state 02). Both episodes come from the original full campaign, before the targeted revisions. Four uniformly spaced recorder indices are shown (two-action recording stride). Frames are vertically flipped for upright display; left/right is preserved.

TABLE V

TERMINAL CONTROLLER STATE AND INDEPENDENTLY RECORDED EXTERNAL EVER-SUCCESS. A CONTROLLER FAILURE CAN FOLLOW AN EARLIER BENCHMARK SUCCESS.

Controller terminal state	External success	External failure
Succeeded	337	7
Failed	26	19
Timeout	11	0

TABLE VI

OUTCOME GROUPS AMONG EXTERNAL FAILURES. CATEGORIES ARE DERIVED FROM RECORDED CONTROLLER MESSAGES WITH A FIXED PRECEDENCE RULE.

Recorded outcome group	Episodes
Phase timeout	13
Controller completion / external failure	7
Grasp / retention rejection	3
Other controller failure	2
Sensor verification rejection	1

E. Remaining Failure Categories

The 26 external failures are grouped by their recorded terminal state and message in Table VI. Phase timeouts form the largest group. The grouping first identifies controller completion with external failure and global-budget exhaustion, then classifies remaining messages as phase timeouts, grasp or retention rejections, sensor-verification rejections, or other failures. All unsuccessful episodes remain in the analysis.

These categories locate where execution stopped. They do not identify physical root causes: a timeout can arise from an inaccessible target, occlusion, calibration error, or contact resistance, and a rejected verification can reflect either placement error or uncertain perception. The task-specific inspections above use additional geometric traces and recordings to examine particular failures.

F. Execution Cost

Long episodes use 324.5 actions on average, compared with 195.3 for Object (Table II). These counts extend to the controller’s stopping condition or the budget, so they measure the cost of executed behavior rather than time to first

success. The median episode across the targeted retests takes 69.6 seconds, and the two dispatcher durations sum to 24.5 minutes. The combined record set has no single campaign wall time. The records contain 0 runtime exceptions. Elapsed times include perception, simulation, rendering, and video overhead, with hardware and concurrency differing across source runs. They describe this evaluation environment and do not measure real-robot control frequency.

VI. DISCUSSION AND LIMITATIONS

ANCHOR makes the information used by each skill explicit. In the two revision cases, a release condition and a post-handle motion target became specific points for intervention. This is the practical value of the representation: it exposes where the controller’s geometric assumptions meet the observed interaction. The evidence supports this use for inspection and targeted revision within the evaluated setting.

a) *Prior knowledge and development effort*: The system’s compact neural component relies on calibrated RGB-D, public asset textures and dimensions, robot geometry, a constrained grammar, and manually developed skills.

Removing demonstration-based action-policy training shifts work to these representations and procedures. Parameter count and weight-file size quantify the detector, but do not quantify total engineering cost, peak memory, or end-to-end latency. Debugging effort was not compared with that of a learned policy.

b) Evaluation scope: The study covers known assets and instruction forms in one simulated embodiment. It does not test unseen assets, unrestricted language, perturbed calibration, or physical hardware. LIBERO was introduced for lifelong robot learning [7]; the present evaluation characterizes manipulation execution and does not measure transfer, continual learning, or forgetting. The targeted initializations informed development. Their complete retests document task-specific changes, while the combined result spans 3 controller versions. A full-suite result for the latest controller remains unmeasured. No component ablations establish the individual contribution of the gallery, offset handling, or local guards.

c) Execution and measurement limits: Clipped commands, pose tolerances, and bounded retries are operational controls, not guarantees of tracking accuracy, collision avoidance, or contact safety. Small task samples can conceal rare failures, and the observed disagreement between controller and evaluator shows why stopping conventions matter. Recording first-success time and final-state success separately would distinguish goal attainment from goal preservation.

d) Reproducibility: The artifacts preserve source and resource checksums, official initial-state identifiers, per-record controller versions, original episode records, and the scripts that generate the reported tables and figures. These establish the provenance of the measurements. Observation restrictions are supported by interface and code inspection; implementation-generated boundary counters are not independent attestations of process isolation.

VII. CONCLUSION

We presented ANCHOR, an asset-informed system that connects language instructions to geometric manipulation skills through observed scene geometry, carried-object off-sets, and local transition checks. One frozen 172M-parameter detector supplies its neural component, with no robot-demonstration training of an action policy. The original complete evaluation records 360/400 successes; targeted microwave and drawer revisions record 8/10 and 9/10. The combined 93.5% summary spans 3 controller versions and retains that distinction. These results characterize how explicit geometric execution supports manipulation and targeted skill revision under known-asset and calibrated RGB-D assumptions.

ACKNOWLEDGMENTS AND AI DISCLOSURE

OpenAI GPT-6-Astra (via Codex) assisted in drafting and editing the text in all sections, preparing the LaTeX source, and writing scripts for the numerical tables and figures. Results for ANCHOR are computed from existing simulator evaluation records; comparison values are taken from the cited publications. The rollout images are extracted from our recordings; no generative image model was used.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024, version 3, Appendix E, Table 12.
- [2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025, version 1, Table 2.
- [3] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025, version 2, Table I.
- [4] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.
- [5] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composible 3d value maps for robotic manipulation with language models,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 540–562.
- [6] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” *arXiv preprint arXiv:2409.01652*, 2024.
- [7] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [8] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *arXiv preprint arXiv:2010.01083*, 2020.
- [9] L. Manuelli, W. Gao, P. Florence, and R. Tedrake, “kPAM: Keypoint affordances for category-level robotic manipulation,” *arXiv preprint arXiv:1903.06684*, 2019.
- [10] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [11] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision*, 2024.
- [12] OpenVLA Contributors, “LIBERO evaluation script and environment utilities,” <https://github.com/openvla/openvla>, accessed September 2026. Paths: experiments/robot/libero/run_libero_eval.py and libero_utils.py.
- [13] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [14] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” *arXiv preprint arXiv:2009.12293v3*, 2025.